

How a Transformer Works

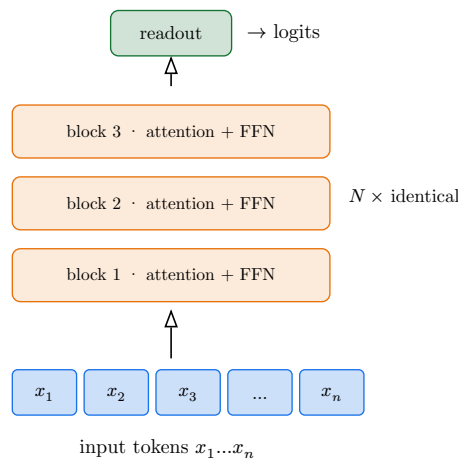
Attention, multi-head, blocks, positional information, and how to read the output

architecture primer · src/transformer.py · src/layers.py · src/cot.py · 2026-07-07

In one sentence. A transformer turns a set of token vectors into a better set of token vectors by repeating one idea — **let every token gather information from every other token, weighted by how relevant they are** (attention), then **let each token think on its own** (a small MLP) — and stacks that idea many times, reading the answer off whichever token(s) you choose.

1 The shape of a transformer

A transformer operates on a **sequence (or set) of tokens**, each a vector of width d . Everything in between the input and the output is a stack of identical **blocks**; each block has two sub-layers — **multi-head self-attention** (tokens exchange information) and a **feed-forward network** (each token is transformed on its own). The output is read from one or more tokens, depending on the task.



Two facts shape everything else:

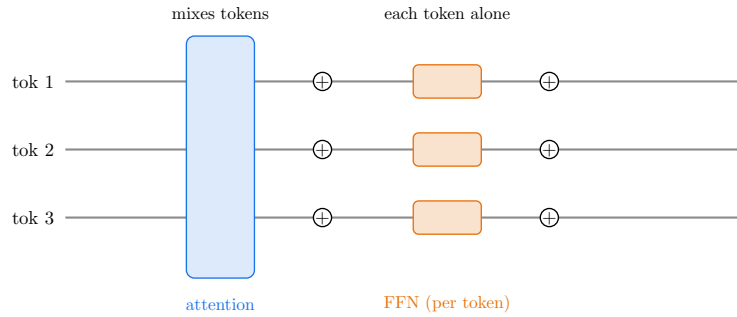
Tokens never disappear the block maps n tokens to n tokens of the same width. Information accumulates in a **residual stream** — each sub-layer **adds** to the running token vectors rather than replacing them.

Attention is permutation-equivariant shuffle the input tokens and the output is the same, just shuffled. The architecture has **no built-in notion of order or position** — that has to be supplied separately (§9).

2 The residual stream: a shared workspace

Picture each token as a horizontal “wire” carrying a d -dimensional vector through the depth of the network. A block never overwrites a wire; it computes an update and **adds** it on:

$$x \leftarrow x + \text{Attention}(x), \quad x \leftarrow x + \text{FFN}(x).$$



This split is the whole division of labour: **attention is the only place tokens talk to each other**; the **FFN is the only place a token is nonlinearly transformed**. Stacking blocks alternates “communicate” and “compute” — and the additive stream means gradients flow cleanly to the bottom, which is what lets transformers be very deep.

3 Self-attention: the core mechanism

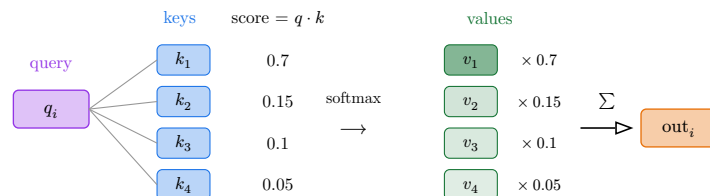
The intuition is a **soft, content-based lookup**. Each token asks a question (a **query**), every token advertises a label (a **key**), and carries a payload (a **value**). A token’s output is the average of all payloads, weighted by how well each key answers its query.

From each token vector x_i we make three projections:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i.$$

Token i scores every token j by the dot product $q_i \cdot k_j$ (high when query and key point the same way), turns the scores into weights with a softmax, and returns the weighted sum of values:

$$\alpha_{ij} = \text{softmax}_j \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right), \quad \text{out}_i = \sum_j \alpha_{ij} v_j.$$

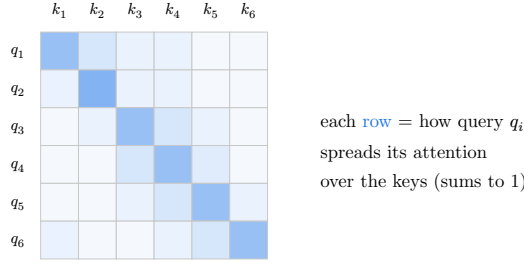


The output token is a **blend of the value vectors of whichever tokens it found relevant**. Nothing about which tokens are relevant is fixed — it is recomputed from the content every forward pass. That dynamism is the source of the transformer’s power.

3.1 The whole thing at once: the attention matrix

Stacking all queries and keys, the scores form an $n \times n$ matrix; the softmax is taken along each row, so **every row sums to 1**. It is a data-dependent “mixing matrix” that says how much each token reads from each other token:

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right)}_{[n \times n] \text{ weights}} V.$$



4 Why attention is so useful

property	why it matters
content-based	which tokens interact is decided by the data, not by a fixed kernel or a fixed graph — the model learns what to look at ,
global in one step	any token can read any other in a single layer; long-range dependencies do not have to be relayed hop-by-hop (unlike RNNs or message passing),
dynamic routing	the mixing matrix is recomputed per input, so the same weights handle very different structures,
permutation-equivariant	it acts on a set ; perfect for tokens, pixels, or graph nodes with no natural order,
parallel	all positions are processed at once (no sequential recurrence), which is what makes training on modern hardware fast.

The contrast with earlier architectures is the point: a convolution mixes a **fixed local window**, an RNN mixes **sequentially through time**, a message-passing GNN mixes **along fixed edges**. Attention mixes **all-pairs, content-dependently, in parallel**. (In this codebase, GlobalAttnConv in `src/layers.py` is exactly all-pairs attention over graph nodes — a graph “conv” with no edge restriction.)

4.1 A detail that matters: the $1/\sqrt{d_k}$ scale

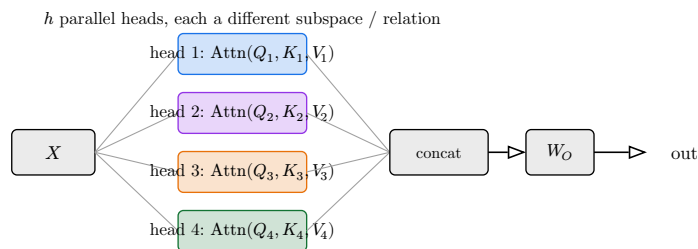
Dot products of two d_k -dimensional vectors grow in magnitude like $\sqrt{d_k}$. Left unscaled, large scores push the softmax into a near one-hot spike with vanishing gradients. Dividing by $\sqrt{d_k}$ keeps the scores in a sane range so the softmax stays soft and trainable. Small, but the model does not learn well without it.

5 Multi-head attention

One attention operation can only express **one** notion of relevance at a time. **Multi-head** attention runs several attentions in parallel, each in its own learned subspace, so different heads can specialise — one might track syntactic neighbours, another long-range coreference, another positional offset.

Split the width d into h heads of size d/h ; each head has its own W_Q, W_K, W_V ; attend independently; concatenate; project back to d :

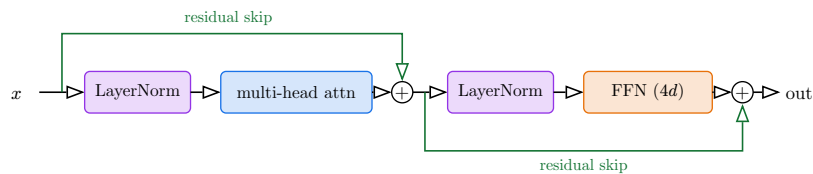
$$\text{head}_\ell = \text{Attention}(XW_Q^\ell, XW_K^\ell, XW_V^\ell), \quad \text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O.$$



Crucially the heads cost almost nothing extra: h heads of size d/h have the same total parameter count and FLOPs as one head of size d . You get the diversity of several attention patterns for the price of one. (In `_EncoderBlock`, this is the single `nn.MultiheadAttention(dim, heads, ...)` call; in `GlobalAttnConv` it is the explicit `[N, N, H]` score tensor.)

6 The transformer block

Each block wraps the two sub-layers with **layer normalization** and **residual connections**, and the FFN is a small two-layer MLP that widens to $4d$ and back. Modern transformers put the norm **before** each sub-layer (**pre-norm**), which trains far more stably at depth.



This is exactly `_EncoderBlock.forward` in `src/transformer.py`:

```
h = self.norm1(x); a = self.attn(h, h, h, ...); x = x + a # attention sub-layer
h = self.norm2(x); x = x + self.ffn(h)                 # feed-forward sub-layer
```

The residual adds are **not optional decoration** — they are what carries the original signal forward and keeps gradients healthy, which is why every transformer has them regardless of how the config exposes other knobs.

7 Positional information

Because attention is permutation-equivariant, a plain transformer cannot tell “dog bites man” from “man bites dog” — it sees a **set**. Position must be injected. The main families:

method	idea
absolute (sinusoidal / learned)	add a position vector to each token embedding (original Transformer, BERT, GPT),
relative	bias attention by the offset $i - j$ rather than absolute index — generalises better across lengths,
RoPE / ALiBi	rotate queries/keys by position, or add a distance-decaying bias to the scores — the dominant choices in modern LLMs,
attention bias	add a learned bias per relation directly to the score matrix (Graphormer). In <code>GlobalAttnConv</code> this is the shortest-path-distance bias <code>spd_bias</code> ,
graph Laplacian PE	for graphs, use Laplacian eigenvectors as positional coordinates — see the companion <code>laplacian-eigenvectors.typ</code> .

8 Masking: who is allowed to attend to whom

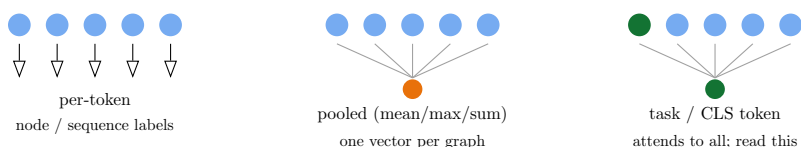
The attention matrix can be **masked** (set entries to $-\infty$ before the softmax) to forbid certain token pairs. Two patterns dominate, and structured masks build the rest:



Bidirectional (encoder) lets every token use full context — right for classification and for our graph tasks. **Causal** (decoder) forbids looking ahead — required for left-to-right generation, where the model must predict token t from only $1 \dots t - 1$. Padding masks hide filler tokens in ragged batches; and arbitrary **structured** masks encode task rules — e.g. the scratchpad mask in `cot-tokens.typ`, where the graph is read-only, the scratchpad is causal, and a task token sees everything.

9 How to read the output

A block returns n refined token vectors. **Which** of them you turn into a prediction is the readout, and it depends on the task:



Per-token feed each token to a shared classifier. Used for sequence labelling and **node classification** (our `task: node`).

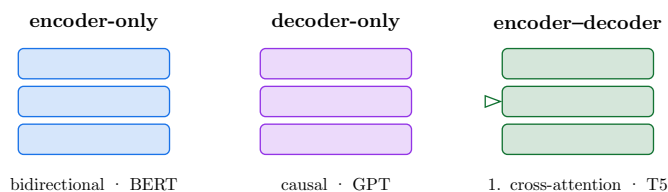
Pooling average / max / sum all tokens into one vector, then classify. Used for sentence- and **graph-level** tasks (pooling: mean on the node path).

CLS / task token prepend (or append) one special learnable token whose only job is to gather a summary via attention; read the answer off it. This is the **task token** in our `node_edge` and CoT models.

Autoregressive in a decoder, the output at each position predicts the **next** token; generation feeds predictions back in one at a time.

10 Families of transformers

The same block composes into three architectures, distinguished by masking and by whether two sequences are involved:



Encoder-only (BERT-style) bidirectional; great for understanding/classification. **Our graph models are encoders.**

Decoder-only (GPT-style) causal; generates text token by token. The dominant LLM design.

Encoder-decoder (T5-style) an encoder reads the input, a decoder generates the output and pulls from the encoder via **cross-attention** (queries from the decoder, keys/values from the encoder).

10.1 Attention variants (beating the $O(n^2)$ cost)

Full attention compares every pair, so it costs $O(n^2d)$ time and memory — fine for hundreds of tokens, painful for very long sequences. The efficiency literature trades exactness for scale:

variant	idea
local / windowed	each token attends only to a nearby window (Longformer, sliding-window),
sparse	attend to a fixed sparse pattern + a few global tokens (BigBird, Sparse Transformer),
linear / kernel	approximate softmax so attention is $O(n)$ (Performer, Linear Transformer),
low-rank	project keys/values to a small set before attending (Linformer),

variant	idea
flash attention	exact, but a fused kernel that never materialises the $n \times n$ matrix — faster and lower-memory.

For graphs there is a further axis: **graph transformers** either restrict attention to edges, or keep it global and inject structure through positional encodings (Laplacian PE) and attention biases (SPD) — which is exactly the design space these experiments live in.

11 Complexity at a glance

For n tokens of width d with an FFN factor of 4:

sub-layer	cost	dominates when
self-attention	$O(n^2d)$	sequences are long (n large),
feed-forward	$O(nd^2)$	the model is wide (d large),
total per block	$O(n^2d + nd^2)$	

The quadratic-in- n attention term is what the efficient variants above attack; the quadratic-in- d FFN term is usually the bigger share at typical sizes.

12 Encoder vs decoder: the same blocks, one mask apart

A misconception worth killing early: **encoder and decoder are not two architectures**. They are the same block stack — attention + FFN, pre-norm, residuals — used in two different ways. Our own code is the proof: `src/cot.py` builds the decoder-only CoT model out of the **same** `_EncoderBlock` class as every encoder in the repo; its docstring says it in one line — “a causal attention mask is the only difference from the encoder stack.”

What actually differs is three usage choices, not the internals:

	encoder use (our graph models)	decoder use (our CoT model)
mask	bidirectional — every token sees all	causal — token t sees only $1\dots t$
readout	pool / task token $\rightarrow$ one label	every position $\rightarrow$ logits for the next token
at inference	one forward pass, done	a loop: generate a token, feed it back, repeat

The two words describe the two halves of any sequence model’s job. **Encoding** is turning input tokens into contextualized vectors — running the stack. Both families do this; a “decoder” encodes its prompt too. **Decoding** is the extra step only generation has: turning the model’s output distributions back into concrete tokens, one at a time. The names stuck from the encoder–decoder translation models (T5-style, §11), where the two roles live in two separate stacks connected by cross-attention — that is the only family with a genuinely new structural piece. A decoder-**only** model like GPT (or ours) simply does both jobs in one stack: the causal mask means every prefix is encoded, and the next token is decoded from the last position’s logits.

13 Our CoT model concretely: what the tokens are

Everything the AR-CoT model ever sees is a symbol from one small fixed vocabulary (`CoTVocab` in `src/cot_tokens.py`). With `max_nodes: 32` there are exactly 42 symbols:

token id	symbol	meaning
0...31	node ids	one symbol per node; in the WL isomorphism trace these double as colour ids (colours are canonical small ints, so they share the range),
32	PAD	filler for ragged batches — never supervised, never attended,
33	N	opens the node roster <code>N 0 1 2 ...</code> (distractor mass at fixed n),
34	E	opens the edge list; edges follow as bare id pairs <code>u v u v ...</code> ,
35	TRACE	end of prompt — “start computing here”,
36	SEP	separates BFS levels / WL rounds,
37–39	ANS YES NO	the answer slot and the two possible answers,
40	EOS	end of sequence — generation stops here,
41	EXP	marks “now expanding node u ” inside a verbose trace round.

A whole training sequence is prompt + trace + answer, laid end to end. For a 6-node connected graph (backbone 0–1–2–3, leaves 4 on 1 and 5 on 2), the `bfs_expand` sequence is:

```
N 0 1 2 3 4 5 E 1 4 0 1 2 5 1 2 2 3 TRACE 0 SEP EXP 0 1 SEP EXP 1 2 4 SEP EXP 2 3 5 EXP 4 SEP EXP
3 EXP 5 ANS YES EOS
```

prompt: the graph, edges in shuffled order · trace: BFS frontier by frontier, EXP u then u 's new children · answer

Three things to notice. First, **there is no other input channel**: no adjacency matrix, no node features — the graph exists only as this token string, and any “understanding” of it must be built by attention over these symbols. Second, the trace is the **supervised intermediate computation**: the model is not asked to leap from edge list to YES/NO; it is taught to write out BFS (or WL colour refinement) step by step, and the answer is only the last three tokens. Third, node ids are randomly permuted per graph, so no id pattern leaks the answer — the model must actually read the edge list.

14 Two regimes: teacher forcing (training) vs greedy decoding (eval)

The same model is used completely differently at train and eval time, and the distinction explains every metric in our logs.

14.1 Training: teacher forcing — all positions at once, one update per batch

During training the model **never generates anything**. The full gold sequence (prompt + trace + answer) is fed in one forward pass; thanks to the causal mask, the output at position t is a prediction of token $t + 1$ made from only $1...t$ — so **every position of every sequence is a separate next-token exercise, graded simultaneously**. The loss is cross-entropy on the completion positions (the trace and, weighted by `answer_loss_weight`, the answer slot; prompt and padding are ignored), and there is one optimizer update per **batch** — at batch 64 and 25,600 training sequences, that is 400 updates per epoch, not one.

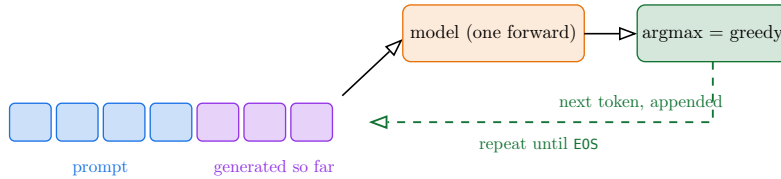
This is called **teacher forcing** because at every position the model predicts from the **gold** prefix — as if a teacher corrected each token before it writes the next. It is what makes training parallel and fast. The `tf_train` / `tf_test` numbers in the logs are exactly this: next-token accuracy given a gold prefix.

14.2 Eval: greedy decoding — the model on its own

“Decoding” answers the question the forward pass leaves open: the model outputs a **distribution** over 42 symbols at each position — which token do we actually commit to? A **decoding strategy** turns distributions into a sequence. **Greedy** is the simplest: take the argmax, always. (Sampling, beam

search, etc. are other strategies; for an algorithm trace with a unique correct continuation, greedy is the natural choice.)

Generation is a loop (`generate()` in `src/cot.py`): feed the prompt, take the argmax at the last position, **append it to the sequence, feed the whole thing back in**, and repeat until EOS:



The feedback edge is not an implementation detail — **it is the mechanism this whole thesis measures**. Each generated token re-enters the network and passes through all layers again, so a depth-2 model that emits 300 trace tokens performs 300 sequential computation steps a plain depth-2 encoder simply does not have (Merrill & Sabharwal). That is why the trace model solves diameter-18 connectivity at depth 2 while the answer-only control sits at chance.

It also explains the metric gap in a stalled run: `tf_test` ≈ 1 with `decoded` ≈ 0.5 means the model predicts the next token nearly perfectly **when the prefix is gold**, but during free generation one early trace error puts it on a prefix it was never trained on, errors compound, and it then faithfully “reads off” the wrong answer from its own derailed trace. `decoded` (answer accuracy after greedy decoding) and `trace_em` (exact match of the whole generated trace) are the honest numbers; teacher-forced accuracy is the optimistic one.

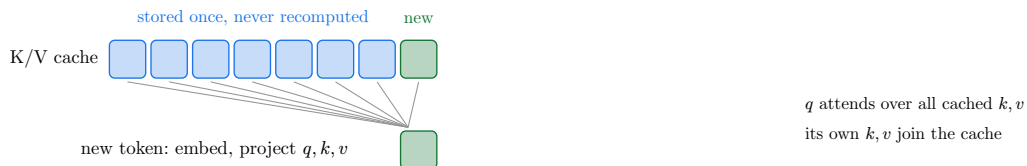
15 K and V are activations; W_K and W_V are the parameters

A confusion worth pinning down precisely, because the words “the K and V matrices” get used for both things. What is **learnable** is the three projection matrices W_Q, W_K, W_V (plus W_O) — fixed after training, the same for every input. What attention actually **uses** are the per-token vectors they produce on each forward pass:

$$k_t = W_K x_t, \quad v_t = W_V x_t$$

— these are **activations**, like any hidden state: recomputed from the data every pass, different for every input, never “learned” or stored in the model.

This distinction is what makes the **KV cache** legitimate during generation. At eval the weights are frozen, and the mask is causal — so token t ’s key and value depend only on tokens $1 \dots t$, which never change as the sequence grows. When we append token 301, tokens 1–300 produce **bit-identical** k, v to the previous step. Recomputing them (which the naive loop above does — a full forward over the whole prefix per generated token) is pure waste:



With the cache (`generate()` since commit `ae6b367`), each step embeds **only the newest token**, projects its q, k, v , appends k, v to the per-layer cache, and attends against the stored keys — the prefix is never touched again. The outputs are token-identical to the naive loop (verified against the trained `hard_diam` checkpoint); only the cost changes: per generated token, a full re-forward costs $O(L)$ FFN/projection work versus $O(1)$ for the cached step, so the saving grows with trace length. Training is unaffected — teacher forcing is already one parallel pass and needs no cache.

16 Summary

Concept	What to remember
token + residual stream	tokens are vectors carried through depth; sub-layers add updates, never overwrite,
self-attention	content-based soft lookup: $\text{softmax}(QK^\top/\sqrt{d_k})V$; rows of the weight matrix sum to 1,
why useful	global, content-based, dynamic, permutation-equivariant, parallel,
multi-head	several attentions in parallel subspaces for the price of one; different heads, different relations,
the block	pre-norm + residual around (attention, then FFN); attention mixes tokens, FFN transforms each,
position	must be added (absolute / relative / RoPE / ALiBi / bias / Laplacian PE) — attention alone is order-blind,
masking	bidirectional vs causal vs structured; controls who attends to whom,
readout	per-token, pooled, or a CLS/task token — pick by task,
families	encoder (understand) · decoder (generate) · encoder-decoder (translate, via cross-attention) — same blocks, different mask and readout ,
teacher forcing	training = one parallel pass over the gold sequence; every position is a next-token exercise; no generation happens,
greedy decoding	eval = argmax, append, feed back, repeat; each fed-back token buys one sequential computation step,
KV cache	W_K, W_V are parameters; k_t, v_t are activations — frozen weights + causal mask make past k, v reusable,
cost	$O(n^2d)$ attention drives the efficient-attention zoo.

A transformer is, at bottom, **one block applied many times**: let tokens gather what they need from each other, let each token think, add it to the stream, repeat — then read the answer off whichever token the task points to.